

Angular Tutorial

A Complete Guide

20 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/angular/>

Getting Started with Angular

What is Angular?

Angular is a TypeScript-based open-source front-end web application framework led by the Angular team at Google. It is designed for building dynamic, single-page applications (SPAs) with a rich, responsive user interface. First released in 2016 as a complete rewrite of AngularJS, Angular provides a comprehensive solution for client-side development including data binding, dependency injection, routing, and form handling out of the box.

Key Features of Angular

- **Component-based architecture** — Build encapsulated, reusable UI pieces.
- **Dependency Injection** — Built-in DI system for manageable, testable code.
- **Powerful templating** — Declarative templates with data binding, directives, and pipes.
- **RxJS integration** — Reactive programming model using Observables.
- **Router** — Feature-rich client-side routing with lazy loading.
- **Forms** — Both template-driven and reactive forms with validation.
- **CLI** — Command-line tool for scaffolding, building, testing, and deploying.

Pros and Cons

Pros

- Full-featured framework — includes routing, HTTP client, forms, and more.

- Strong typing with TypeScript improves code quality and maintainability.
- Excellent tooling with Angular CLI and IDE support.
- Modular architecture promotes separation of concerns.
- Large ecosystem and community support.

Cons

- Steep learning curve compared to lighter frameworks like React or Vue.
- Larger bundle size out of the box.
- Complexity can be overkill for small projects.
- Frequent major version changes require migration effort.

Environment Setup

Before you start, install the following prerequisites:

1. **Node.js** — Download from nodejs.org. Angular requires an active LTS version. Verify with `node --version`.
2. **npm** — Comes with Node.js. Verify with `npm --version`.
3. **Angular CLI** — Install globally: `npm install -g @angular/cli`. Verify with `ng version`.

Angular CLI is the primary tool for creating and managing Angular projects. It handles scaffolding, building, testing, and deployment.

Creating Your First App

Open a terminal and run the following commands:

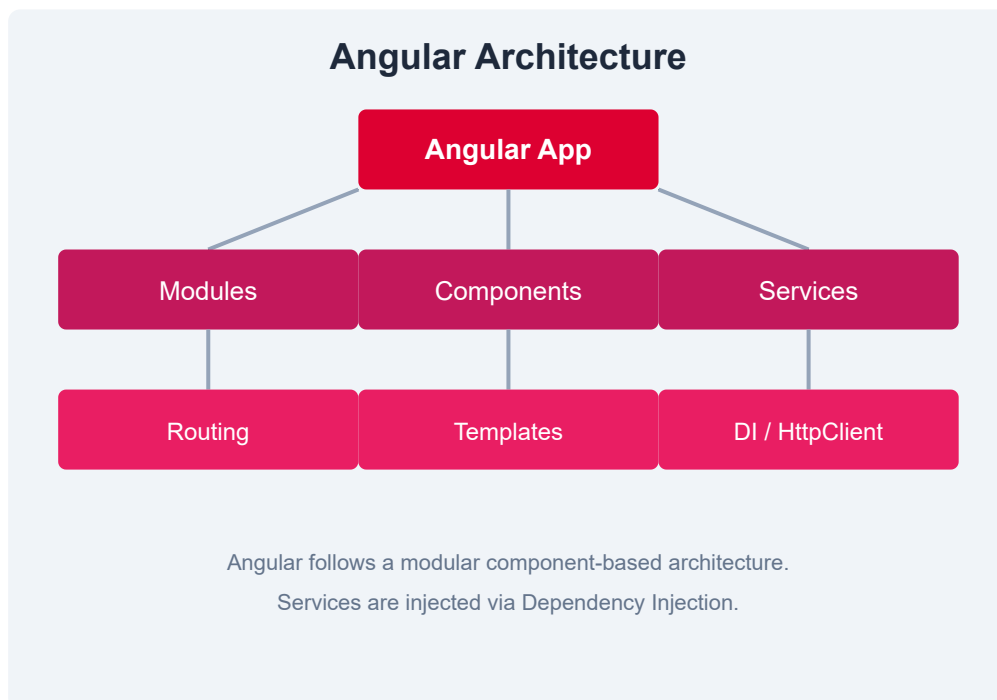
```
ng new my-first-app
cd my-first-app
ng serve
```

The `ng serve` command starts a development server at `http://localhost:4200`. Open that URL in your browser to see your app running. Any changes you make to source files will automatically reload the browser.

Project Structure

Here is the typical structure of an Angular project:

```
my-first-app/  
  src/  
    app/  
      app.component.ts    # Root component class  
      app.component.html  # Root component template  
      app.component.css   # Root component styles  
      app.module.ts       # Root module  
    assets/               # Static files (images, fonts)  
    index.html            # Entry HTML file  
    main.ts               # Bootstrap entry point  
    styles.css            # Global styles  
  angular.json            # CLI configuration  
  package.json            # npm dependencies  
  tsconfig.json           # TypeScript configuration
```



MVC Architecture in Angular

Angular follows a component-based variation of the MVC (Model-View-Controller) pattern. The **Model** is represented by services and data models, the **View** by templates (HTML), and the **Controller** by component classes that

manage the logic and state. Unlike traditional MVC, Angular binds these layers together through dependency injection and data binding, creating a cohesive, testable structure.

Angular's architecture promotes loose coupling between components through well-defined interfaces, making your application easier to maintain and extend over time.

Understanding Modules

Every Angular app has at least one module — the root module (`AppModule`). Modules are decorated with `@NgModule` and declare which components, directives, pipes, and services belong together. They help organize code into cohesive blocks of functionality.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Main Entry Point

The `main.ts` file bootstraps the application by passing the root module to the platform:

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Summary

In this chapter, you learned what Angular is, its core features, how to set up your environment, create a new project, and understand the project structure and MVC architecture. You are now ready to dive into components in Chapter 2.

Exercise 1: Install Node.js and Angular CLI on your machine. Create a new Angular project called `hello-angular` using `ng new`. Run `ng serve`, open `http://localhost:4200`, and verify you see the default Angular welcome page. Then explore the generated project files and identify each part of the MVC pattern.

Components

Component Basics

Components are the fundamental building blocks of an Angular application. Each component consists of a TypeScript class, an HTML template, and optional CSS styles. The `@Component` decorator provides metadata including the selector, template URL or inline template, and style URLs.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  template: `<h1>Hello, {{ name }}!</h1>`,
  styles: [`h1 { color: #dd0031; }`]
})
export class HelloComponent {
  name = 'Angular';
}
```

Component Selector

The `selector` defines how the component is used in HTML. It can be an element selector (`'app-hello'`), an attribute selector (`'[app-hello]'`), or a class selector (`' .app-hello'`). Element selectors are the most common.

Angular components are always used with a custom tag in templates. The selector must be unique across the application to avoid conflicts.

Lifecycle Hooks

Angular components have a well-defined lifecycle. You can hook into key moments by implementing lifecycle hook interfaces:

ngOnInit

Called once after the component is initialized and inputs are bound. Use it for initialization logic, fetching data, or setting up subscriptions.

```
ngOnInit() {  
  this.loadUserData();  
}
```

ngOnChanges

Called whenever input properties change. Receives a `SimpleChanges` object with current and previous values.

```
ngOnChanges(changes: SimpleChanges) {  
  if (changes['userId']) {  
    this.reloadData();  
  }  
}
```

ngAfterViewInit

Called after the component's view (and child views) have been rendered. Safe to access `@ViewChild` elements here.

```
ngAfterViewInit() {  
  console.log('View initialized', this.myElement);  
}
```

ngOnDestroy

Called just before the component is destroyed. Use it to clean up subscriptions, timers, or event listeners to prevent memory leaks.

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
  clearInterval(this.timer);  
}
```

ViewEncapsulation

Angular emulates Shadow DOM to scope component styles. Three strategies are available:

- **Emulated** (default) — Styles are scoped using attribute selectors without native Shadow DOM.
- **ShadowDom** — Uses the browser's native Shadow DOM API for true style isolation.
- **None** — Styles are global and not scoped to the component.

```
@Component({  
  selector: 'app-card',  
  templateUrl: './card.component.html',  
  encapsulation: ViewEncapsulation.ShadowDom  
})
```

@Input and @Output

Components communicate with each other using `@Input` and `@Output` decorators.

@Input

Binds a property to a value passed from a parent component:

```
@Input() userName: string = '';
@Input('aliasTitle') title: string = '';
```

@Output with EventEmitter

Emits events to the parent component:

```
@Output() userSelected = new EventEmitter<User>();

selectUser(user: User) {
  this.userSelected.emit(user);
}
```

The parent listens like this: `<app-user (userSelected)="onUserSelected($event)"></app-user>`.

@ViewChild

Access a child component, directive, or DOM element from the parent component class:

```
@ViewChild('submitBtn') submitButton!: ElementRef;
@ViewChild(ChildComponent) child!: ChildComponent;

ngAfterViewInit() {
  this.submitButton.nativeElement.focus();
  this.child.doSomething();
}
```

Services in Components

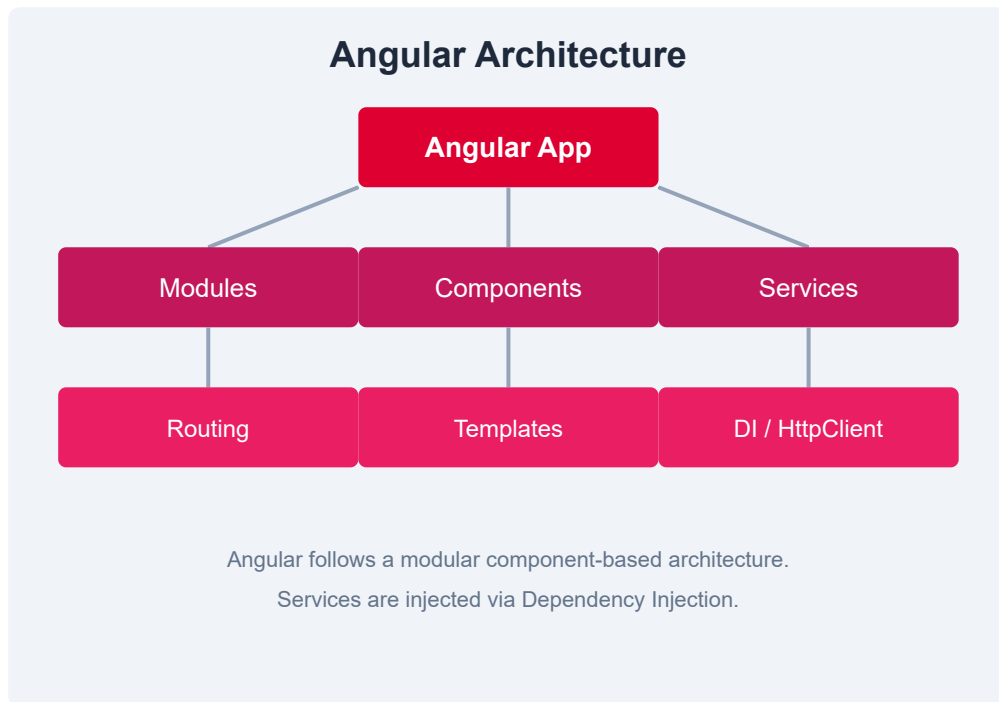
Services provide shared logic, data access, or state management. They are injected into components via the constructor:

```

constructor(private userService: UserService) { }

loadUsers() {
  this.userService.getUsers().subscribe(users => this.users = users);
}

```



Content Projection with ng-content

Project content from a parent into a child component using `<ng-content>`:

```

<!-- card.component.html -->
<div class="card">
  <ng-content select="[card-header]"></ng-content>
  <ng-content select="[card-body]"></ng-content>
</div>

```

Dynamic Components

Load components at runtime using `ComponentFactoryResolver` (legacy) or the newer `ViewContainerRef` approach with standalone components. This is useful for modals, tabs, or dashboards where components are determined dynamically.

In Angular 13+, the `ViewContainerRef.createComponent` method does not require `ComponentFactoryResolver` when used with standalone components.

Angular Elements

Angular Elements allow you to package Angular components as custom elements (Web Components) that can be used in any HTML page, even outside Angular applications. Use `@angular/elements` and the `createCustomElement` function.

Exercise 2: Create a `UserCardComponent` with `@Input()` `user` and `@Output()` `delete`. Add lifecycle hooks: `ngOnInit` to log user data, and `ngOnDestroy` to log cleanup. Use content projection to inject a custom footer into the card. Use the component in your app with a list of users and handle the delete event.

Templates & Binding

Template Syntax

Angular templates are HTML-based with added syntax for data binding, directives, and template variables. Templates can be inline (defined with the `template` property) or external (using `templateUrl`). They support standard HTML elements, Angular components, and special Angular constructs.

Template Statements

Template statements are methods or properties in the component class that respond to events raised in the template. They are written within parentheses `()` on the right side of an event binding. Template statements can call methods, assign values, or use basic operators.

```
<button (click)="save($event)">Save</button>  
<input (keyup.enter)="search(term.value)" #term>
```

Template Reference Variables (#)

Template reference variables provide a way to access a DOM element, directive, or component directly within the template. They are declared with the `#` prefix and can be used anywhere in the same template.

```
<input #phone placeholder="Phone number">  
<button (click)="call(phone.value)">Call</button>
```

Template reference variables are scoped to the template they are declared in. They are resolved before any structural directive context variables.

SVG Templates

Angular supports SVG in templates, allowing you to create dynamic, data-driven graphics. You can bind attributes, events, and use Angular directives within SVG elements just like HTML.

```
<svg width="200" height="200">
  <circle [attr.cx]="cx" [attr.cy]="cy" r="20"
    (click)="moveCircle()" fill="#dd0031"/>
</svg>
```

Data Binding Overview

Angular provides four primary forms of data binding, each with a distinct direction of data flow:

- **Interpolation** — from component to template (one-way).
- **Property binding** — from component to template (one-way).
- **Event binding** — from template to component (one-way).
- **Two-way binding** — bidirectional with `[(ngModel)]`.

Interpolation {{ }}

Interpolation embeds component property values into the template. Angular evaluates the expression between double curly braces and converts it to a string.

```
<p>Welcome, {{ username }}!</p>
<p>You have {{ cart.length }} items in your cart.</p>
<p>Total: {{ price * quantity | currency }}</p>
```

Event Binding

Event binding listens to DOM events and executes template statements. Target events are wrapped in parentheses.

```
<button (click)="toggle()">Toggle</button>
<form (ngSubmit)="onSubmit()" #myForm="ngForm">
<input (keyup.escape)="clearInput()">
```

Property Binding [property]

Property binding sets an element property to a component value. Use square brackets around the property.

```
<img [src]="user.avatarUrl">
<button [disabled]="isSaving">Save</button>
<app-child [user]="selectedUser"></app-child>
```

Attribute Binding [attr.]

Use attribute binding when an element does not have a corresponding DOM property, such as ARIA attributes, colspan, or SVG attributes.

```
<td [attr.colspan]="colspan">Merged cell</td>
<div [attr.aria-label]="labelText">Content</div>
```

Class Binding [class.name]

Dynamically add or remove CSS classes:

```
<div [class.active]="isActive">Active item</div>
<div [class.valid]="isValid" [class.invalid]="!isValid">Status</div>
<div [class]="dynamicClasses">Multiple classes</div>
```

Style Binding [style.prop]

Set inline styles dynamically. Supports units like px, em, and %.

```
<div [style.color]="textColor">Styled text</div>
<div [style.font-size.px]="fontSize">Font sized</div>
<div [style.background]="bgColor">Background</div>
```

Two-Way Binding [(ngModel)]

Two-way binding combines property binding and event binding into one convenient syntax using `[(ngModel)]`. It requires `FormsModule` to be imported.

```
<input [(ngModel)]="username" placeholder="Username">
<p>Hello, {{ username }}!</p>
```

Under the hood, `[(ngModel)]` is syntactic sugar for `[ngModel]="username" (ngModelChange)="username=$event"`.

 Angular data binding diagram

Summary

You have learned the full spectrum of Angular template syntax and data binding: interpolation, event binding, property binding, attribute binding, class binding, style binding, and two-way binding. These binding mechanisms form the communication backbone between your component logic and the user interface.

Exercise 3: Create a simple profile editor component. Use interpolation to display a user's name and bio. Use property binding for the avatar image. Use event binding on an "Edit" button to toggle editing mode. Use two-way binding on input fields to edit the name and bio. Use class binding to highlight the active field. Display a character count using style binding that turns red when the bio exceeds 200 characters.

Directives

What Are Directives?

Directives are classes that add behavior to elements in your Angular templates. There are three categories: **components** (directives with a template), **attribute directives** (change appearance or behavior of an element), and **structural directives** (change the DOM layout by adding or removing elements).

Attribute Directives

Attribute directives modify the appearance or behavior of an element. Angular provides built-in attribute directives and you can create custom ones.

ngClass

Add or remove multiple CSS classes dynamically:

```
<div [ngClass]="{  
  'highlight': isHighlighted,  
  'error': hasError,  
  'disabled': isDisabled  
}">Conditional classes</div>
```

ngStyle

Set multiple inline styles dynamically:

```
<div [ngStyle]="{
  'color': textColor,
  'font-size': fontSize + 'px',
  'background': bgColor
}">Dynamic styles</div>
```

Custom Attribute Directive

Create a custom directive using the `@Directive` decorator:

```
import { Directive, ElementRef, Renderer2, HostListener, HostBinding } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostBinding('style.backgroundColor') bgColor = 'transparent';

  constructor(private el: ElementRef, private renderer: Renderer2) { }

  @HostListener('mouseenter') onMouseEnter() {
    this.bgColor = 'yellow';
    this.renderer.setStyle(this.el.nativeElement, 'fontWeight', 'bold');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.bgColor = 'transparent';
    this.renderer.setStyle(this.el.nativeElement, 'fontWeight', 'normal');
  }
}
```

Use it in templates: `<p appHighlight>Hover over me!</p>`.

ElementRef gives access to the native DOM element. **Renderer2** is the preferred API for modifying the DOM because it works in both browser and server environments. **@HostListener** listens to events on the host element. **@HostBinding** binds a class or property on the host element.

Structural Directives

Structural directives change the structure of the DOM by adding, removing, or manipulating elements. They are prefixed with an asterisk `*` in templates, which is syntactic sugar for an `<ng-template>` wrapper.

ngIf

Conditionally include a template based on a boolean expression:

```
<div *ngIf="isLoggedIn; else loginPrompt">
  Welcome, {{ username }}!
</div>
<ng-template #loginPrompt>
  <p>Please log in.</p>
</ng-template>
```

ngFor

Iterate over a collection and render a template for each item:

```
<ul>
  <li *ngFor="let user of users; let i = index; trackBy: trackByUserId">
    {{ i + 1 }}. {{ user.name }}
  </li>
</ul>
```

Local variables available: `index`, `first`, `last`, `even`, `odd`, `count`.

Always use `trackBy` with `ngFor` when working with large lists. It helps Angular identify which items changed, improving rendering performance by avoiding unnecessary DOM manipulations.

ngSwitch

Switch between multiple conditional templates:

```
<div [ngSwitch]="role">
  <p *ngSwitchCase="'admin'">Admin Panel</p>
  <p *ngSwitchCase="'editor'">Content Editor</p>
  <p *ngSwitchCase="'viewer'">Read Only</p>
  <p *ngSwitchDefault>Unknown role</p>
</div>
```

Custom Structural Directive

Create your own structural directive using `TemplateRef` and `ViewContainerRef`:

```
import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';

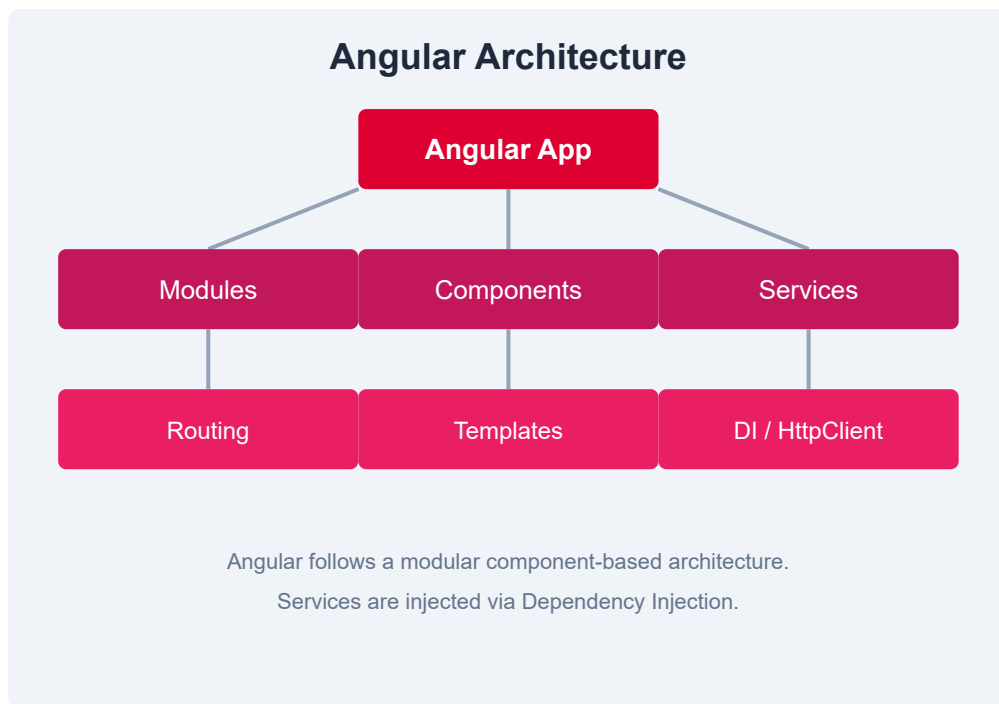
@Directive({
  selector: '[appRepeat]'
})
export class RepeatDirective {
  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) { }

  @Input() set appRepeat(times: number) {
    this.viewContainer.clear();
    for (let i = 0; i < times; i++) {
      this.viewContainer.createEmbeddedView(this.templateRef);
    }
  }
}
```

Usage: `<p *appRepeat="5">Repeated paragraph</p>`.

Microsyntax

Structural directives use microsyntax — a compact language for expressing template logic. The `*ngFor="let item of items; let i = index"` is shorthand that Angular desugars into `<ng-template ngFor let-item="$implicit" [ngForOf]="items" let-i="index">`.



Summary

This chapter covered attribute directives (`ngClass`, `ngStyle`, custom) and structural directives (`ngIf`, `ngFor`, `ngSwitch`, custom). You learned how to use `ElementRef`, `Renderer2`, `@HostListener`, `@HostBinding`, `TemplateRef`, `ViewContainerRef`, and microsyntax to build powerful, reusable behavior.

Exercise 4: Build a custom attribute directive `appTooltip` that shows a tooltip on hover with a configurable text and position (top, bottom, left, right). Include a fade-in animation using `Renderer2`. Then build a custom structural directive `appPermission` that conditionally renders content based on a user role string input.

Pipes

What Are Pipes?

Pipes transform data displayed in templates. They take an input value and optionally arguments, apply formatting or transformation logic, and return the output for display. Angular provides many built-in pipes, and you can create custom pipes for domain-specific needs.

Syntax: `{{ expression | pipeName:arg1:arg2 }}`

Built-in Pipes

DatePipe

Formats date values according to locale rules:

```
<p>{{ today | date }}</p>
<p>{{ today | date:'medium' }}</p>
<p>{{ today | date:'yyyy-MM-dd' }}</p>
<p>{{ today | date:'fullDate' }}</p>
```

UpperCasePipe and LowerCasePipe

Transform text to upper or lower case:

```
<p>{{ 'hello world' | uppercase }}</p>
<p>{{ 'HELLO WORLD' | lowercase }}</p>
```

CurrencyPipe

Formats a number as currency with locale-aware symbol placement:

```
<p>{{ 99.99 | currency }}</p>
<p>{{ 99.99 | currency: 'EUR' }}</p>
<p>{{ 99.99 | currency: 'INR': 'symbol': '₹' }}</p>
```

PercentPipe

Formats a number as a percentage:

```
<p>{{ 0.25 | percent }}</p>
<p>{{ 0.1234 | percent: '2.1-2' }}</p>
```

JsonPipe

Converts an object to its JSON representation for debugging:

```
<pre>{{ user | json }}</pre>
```

AsyncPipe

Unwraps Observables and Promises directly in templates, automatically subscribing and unsubscribing:

```
<p>{{ observableData$ | async }}</p>
<ul>
  <li *ngFor="let item of items$ | async">{{ item.name }}</li>
</ul>
```

The AsyncPipe is a powerful tool for handling asynchronous data without manual subscription management. It automatically unsubscribes when the component is destroyed, preventing memory leaks.

DecimalPipe

Formats numbers with decimal places and grouping separators:

```
<p>{{ 3.14159 | number:'1.2-2' }}</p>
<p>{{ 1234567.89 | number:'5.2-2' }}</p>
```

Chaining Pipes

Pipes can be chained to apply multiple transformations. Each pipe operates on the output of the previous pipe:

```
<p>{{ birthday | date:'fullDate' | uppercase }}</p>
<p>{{ price | currency:'USD' | lowercase }}</p>
```

Order matters — the leftmost pipe is applied first, and its result is passed to the next pipe.

Custom Pipes

Create a custom pipe using the `@Pipe` decorator and implementing the `PipeTransform` interface:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate'
})
export class TruncatePipe implements PipeTransform {
  transform(value: string, maxLength: number = 50, suffix: string = '...'): string {
    if (!value) return '';
    return value.length > maxLength
      ? value.substring(0, maxLength) + suffix
      : value;
  }
}
```

Usage: `{{ longText | truncate:30:'...' }}`

Pure vs Impure Pipes

Pipes are **pure** by default. A pure pipe only re-evaluates when its input value changes by reference (not mutation). An **impure** pipe re-evaluates on every change detection cycle, which is useful for pipes that track internal state or handle mutable data.

```
@Pipe({
  name: 'filterArray',
  pure: false // Impure pipe - re-evaluates on every change detection
})
export class FilterArrayPipe implements PipeTransform {
  transform(items: any[], searchTerm: string): any[] {
    if (!items || !searchTerm) return items;
    return items.filter(item =>
      item.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }
}
```

Use impure pipes sparingly. Because they run on every change detection cycle, they can cause performance issues with large datasets or complex transformations. Consider using memoization or restructuring your data flow instead.

 Angular pipe transformation

Summary

You now know how to use built-in Angular pipes (date, uppercase, lowercase, currency, percent, json, async, decimal), chain pipes together, and create custom pipes with the PipeTransform interface. You also understand the distinction between pure and impure pipes and when to use each.

Exercise 5: Create a custom pipe called `phoneFormat` that transforms a 10-digit string into a formatted phone number: `(123) 456-7890`. Then create a `searchFilter` pipe (impure) that filters a list of products by name. Use both pipes in a component that displays a searchable, formatted phonebook.

Forms

Two Approaches to Forms

Angular provides two form-building approaches. **Template-driven forms** are declarative and placed mostly in the template. **Reactive forms** are more programmatic, with the form model defined explicitly in the component class. Both approaches handle validation, user input, and submission, but reactive forms offer greater flexibility and testability.

Template-Driven Forms

Template-driven forms rely on directives in the template. You import `FormsModule` and use `ngModel` with two-way binding.

```
import { NgForm } from '@angular/forms';

@Component({...})
export class LoginComponent {
  onSubmit(form: NgForm) {
    console.log('Form submitted:', form.value);
  }
}
```

```
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
  <input name="email" ngModel required email placeholder="Email">
  <input name="password" ngModel required minlength="6" type="password">
  <button type="submit" [disabled]="loginForm.invalid">Login</button>
</form>
```

Validation in Template-Driven Forms

Use standard HTML5 validation attributes like `required`, `minlength`, `maxlength`, `pattern`, and `email`. Angular automatically tracks validation state and adds CSS classes (`ng-valid`, `ng-invalid`, `ng-touched`, `ng-dirty`).

```
<input name="phone" ngModel required pattern="[0-9]{10}" #phone="ngModel">
<div *ngIf="phone.invalid && phone.touched" class="error">
  <span *ngIf="phone.errors?.['required']">Phone is required.</span>
  <span *ngIf="phone.errors?.['pattern']">Enter a valid 10-digit phone.</span>
</div>
```

Reactive Forms

Reactive forms are built programmatically using `FormGroup`, `FormControl`, and `FormArray`. Import `ReactiveFormsModule` and define the form model in the component class.

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

@Component({...})
export class ProfileComponent {
  profileForm = new FormGroup({
    firstName: new FormControl('', [Validators.required, Validators.minLength(3)]),
    lastName: new FormControl('', Validators.required),
    email: new FormControl('', [Validators.required, Validators.email]),
    address: new FormGroup({
      street: new FormControl(''),
      city: new FormControl('')
    })
  });

  onSubmit() {
    if (this.profileForm.valid) {
      console.log('Profile saved:', this.profileForm.value);
    }
  }
}
```

```

<form [formGroup]="profileForm" (ngSubmit)="onSubmit()">
  <input formControlName="firstName" placeholder="First Name">
  <input formControlName="lastName" placeholder="Last Name">
  <input formControlName="email" placeholder="Email">
  <div formGroupName="address">
    <input formControlName="street" placeholder="Street">
    <input formControlName="city" placeholder="City">
  </div>
  <button type="submit" [disabled]="profileForm.invalid">Save</button>
</form>

```

FormBuilder

FormBuilder is a service that provides syntactic sugar for creating form controls and groups:

```

constructor(private fb: FormBuilder) {
  this.profileForm = this.fb.group({
    firstName: ['', [Validators.required, Validators.minLength(2)]],
    lastName: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    address: this.fb.group({
      street: ['',],
      city: ['',]
    })
  });
}

```

Custom Validators

Create custom validation functions for specific business rules:

```
export function forbiddenNameValidator(forbiddenName: RegExp): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const forbidden = forbiddenName.test(control.value);
    return forbidden ? { forbiddenName: { value: control.value } } : null;
  };
}

// Usage:
this.form = this.fb.group({
  username: ['', [Validators.required, forbiddenNameValidator(/admin/i)]]
});
```

Dynamic Forms with FormArray

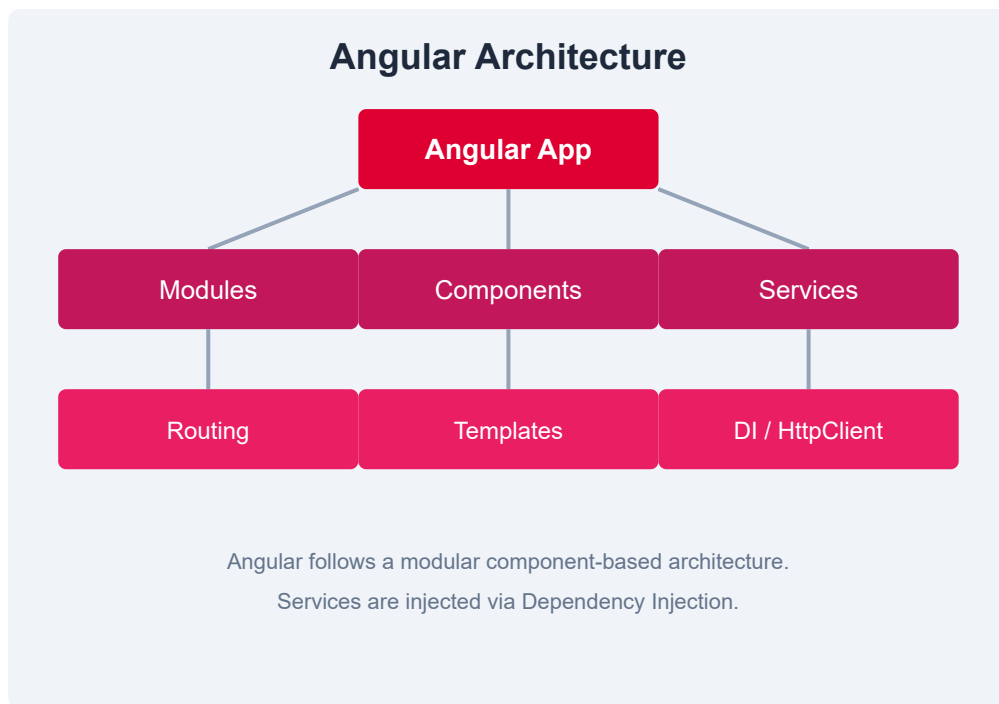
Use `FormArray` to manage a dynamic list of form controls:

```
this.orderForm = this.fb.group({
  items: this.fb.array([this.createItem()])
});

get items() { return this.orderForm.get('items') as FormArray; }

createItem(): FormGroup {
  return this.fb.group({
    product: ['', Validators.required],
    quantity: [1, [Validators.required, Validators.min(1)]]
  });
}

addItem() { this.items.push(this.createItem()); }
removeItem(index: number) { this.items.removeAt(index); }
```



Error Messages

Display user-friendly error messages by checking the control's errors object:

```
<div *ngIf="firstName.invalid && (firstName.dirty || firstName.touched)">
  <p *ngIf="firstName.errors?.['required']">First name is required.</p>
  <p *ngIf="firstName.errors?.['minlength']">
    Minimum {{ firstName.errors?.['minlength'].requiredLength }} characters.
  </p>
</div>
```

Summary

In this chapter, you explored both template-driven and reactive forms, including validation, FormBuilder, custom validators, dynamic FormArrays, and error message display. Reactive forms give you more control and are preferred for complex form scenarios.

Exercise 6: Build a reactive registration form with `firstName`, `lastName`, `email`, `password`, `confirmPassword`, and a dynamic list of phone numbers (`FormArray`). Add custom validators: passwords must match, email cannot be a disposable domain, and each phone number must be valid. Display inline error messages. Disable the submit button while invalid. Log the form value on submit.

Dependency Injection

What is Dependency Injection?

Dependency Injection (DI) is a design pattern where a class receives its dependencies from external sources rather than creating them internally. Angular has a built-in DI framework that provides dependencies to components, directives, pipes, and services. This makes your code more modular, testable, and maintainable.

@Injectable and providedIn

Services are decorated with `@Injectable` to make them available to the DI system. The `providedIn` property specifies where the service is registered:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Singleton – one instance for the entire app
})
export class DataService {
  constructor(private http: HttpClient) { }

  getItems() {
    return this.http.get('/api/items');
  }
}
```

`providedIn: 'root'` creates a tree-shakeable singleton service available application-wide. If the service is never injected, it is tree-shaken from the bundle — reducing the final build size.

DI Patterns

Singleton Service

The most common pattern. A single instance is shared across the application. Use `providedIn: 'root'` or add the service to the `providers` array of the root module.

Factory Providers (`useFactory`)

Create a provider that uses a factory function to determine the dependency at runtime:

```
const loggerFactory = (http: HttpClient) => {
  return environment.production
    ? new RemoteLoggerService(http)
    : new ConsoleLoggerService();
};

@NgModule({
  providers: [
    { provide: LoggerService, useFactory: loggerFactory, deps: [HttpClient] }
  ]
})
```

`useClass`

Substitute one implementation with another:

```
{ provide: ApiService, useClass: MockApiService }
```

`useExisting`

Alias an existing provider to another token:

```
{ provide: OldLoggerService, useExisting: LoggerService }
```

useValue

Provide a static value or configuration object:

```
{ provide: 'API_URL', useValue: 'https://api.example.com/v2' }
```

Hierarchical DI

Angular's DI is hierarchical. Each component can have its own injector that inherits from its parent. This allows different instances of a service at different levels of the component tree.

```
@Component({  
  selector: 'app-child',  
  providers: [DataService] // New instance for this component and its children  
})
```

If a component does not define its own provider, it inherits from the parent injector, eventually reaching the root injector.

DI Decorators

@Optional

Marks a dependency as optional. If the dependency is not found, Angular injects `null` instead of throwing an error:

```
constructor(@Optional() private logger?: LoggerService) { }
```

@Self

Restricts resolution to the current component's injector only — does not check parent injectors:

```
constructor(@Self() private localService: LocalService) { }
```

@SkipSelf

Skips the current component's injector and resolves from the parent injector:

```
constructor(@SkipSelf() private parentService: ParentService) { }
```

@Host

Limits resolution to the host component's injector. Useful when a component is projected into another component via `ng-content`:

```
constructor(@Host() private hostService: HostService) { }
```

 Angular dependency injection hierarchy

Tree-Shakeable Providers

When you use `providedIn: 'root'`, Angular creates a tree-shakeable provider. If the service is never injected, it is removed from the production bundle. This is the recommended approach for most application-wide services.

```
@Injectable({
  providedIn: 'platform' // Singleton across multiple Angular apps on the same page
})
export class SharedService { }
```

Available options: `'root'`, `'platform'`, `'any'`, or a specific module.

Injecting Services into Services

Services can inject other services through their constructor:

```
@Injectable({ providedIn: 'root' })
export class OrderService {
  constructor(
    private http: HttpClient,
    private authService: AuthService,
    private logger: LoggerService
  ) { }
}
```

Summary

You learned about Angular's dependency injection system, including `@Injectable`, `providedIn`, various provider patterns (`useClass`, `useExisting`, `useValue`, `useFactory`), hierarchical injectors, DI decorators (`@Optional`, `@Self`, `@SkipSelf`, `@Host`), and tree-shakeable providers. These tools help you design loosely coupled, testable services.

Exercise 7: Create a `ConfigService` with `providedIn: 'root'` that reads environment-specific settings. Create a `FeatureFlagService` that depends on `ConfigService`. Use `useFactory` to conditionally provide either a live `AnalyticsService` or a no-op `MockAnalyticsService` based on the environment. Use `@Optional` to inject a `CacheService` that might not be provided.

Routing

What is Angular Routing?

The Angular Router enables navigation between different views or components in a single-page application. It interprets browser URLs and maps them to the corresponding component views. The router supports deep linking, lazy loading, guards, resolvers, and nested routes.

RouterModule.forRoot

Configure the router at the application root level using `RouterModule.forRoot` in the root module (or `provideRouter` for standalone apps):

```
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'products', component: ProductListComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Use `RouterModule.forChild(routes)` in feature modules to register child routes without re-creating the router service.

Route Configuration

Each route is an object with several properties:

```
const routes: Routes = [  
  { path: '', component: HomeComponent, pathMatch: 'full' },  
  { path: 'products', component: ProductListComponent },  
  { path: 'products/:id', component: ProductDetailComponent },  
  { path: 'category/:categoryName', component: CategoryComponent },  
  { path: '**', component: NotFoundComponent } // Wildcard – must be last  
];
```

redirectTo

Redirect from one path to another:

```
{ path: 'old-products', redirectTo: '/products', pathMatch: 'full' }
```

Wildcard Route (**)

Catches all unmatched routes — typically used for a 404 page. Must be defined last in the routes array.

Parameterized Routes (:id)

Pass dynamic values via the URL path:

```
// Route config
{ path: 'users/:userId', component: UserDetailComponent }

// Component
import { ActivatedRoute } from '@angular/router';

ngOnInit() {
  this.route.params.subscribe(params => {
    this.userId = params['userId'];
    this.loadUser();
  });
}
```

Query Parameters and Fragments

Access query parameters (`?page=1&sort=name`) and fragments (`#section`):

```
// Reading
this.route.queryParams.subscribe(params => {
  this.page = params['page'];
});
this.route.fragment.subscribe(f => this.section = f);

// Navigation
this.router.navigate(['/products'], {
  queryParams: { page: 2, sort: 'price' },
  fragment: 'reviews'
});
```

Nested Routes (Children)

Create hierarchies of routes with child components:

```
const routes: Routes = [
  {
    path: 'dashboard',
    component: DashboardComponent,
    children: [
      { path: '', redirectTo: 'overview', pathMatch: 'full' },
      { path: 'overview', component: OverviewComponent },
      { path: 'analytics', component: AnalyticsComponent },
      { path: 'settings', component: SettingsComponent }
    ]
  }
];
```

The parent component must include a `<router-outlet>` to render child routes.

routerLink and routerLinkActive

Navigate using declarative links in templates:

```
<nav>
  <a routerLink="/dashboard" routerLinkActive="active">Dashboard</a>
  <a routerLink="/products" routerLinkActive="active"
    [routerLinkActiveOptions]="{ exact: false }">Products</a>
  <a [routerLink]="['/users', user.id]">User Profile</a>
</nav>
```

`routerLinkActive` adds a CSS class (e.g., `active`) when the current URL matches the link. Use `exact: true` for exact path matching.

Router.navigate (Programmatic Navigation)

Navigate imperatively in your component class:

```

constructor(private router: Router) { }

goToProduct(id: number) {
  this.router.navigate(['/products', id]);
}

goToDashboard() {
  this.router.navigateByUrl('/dashboard');
}

```

Route Guards

Protect routes with guards. Common guard types: `CanActivate`, `CanActivateChild`, `CanDeactivate`, `Resolve`, and `CanLoad`.

```

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) { }

  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/login']);
    return false;
  }
}

// Route config
{ path: 'admin', component: AdminComponent, canActivate: [AuthGuard] }

```

Lazy Loading with loadChildren

Load feature modules on demand to reduce initial bundle size:

```
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
}
```

Angular splits the admin module into a separate chunk, loaded only when the user navigates to `/admin`.

 Angular routing flow diagram

Summary

This chapter covered the Angular Router: route configuration, parameterized routes, query parameters and fragments, nested routes, `routerLink` and `routerLinkActive`, programmatic navigation, route guards (`CanActivate`), and lazy loading with `loadChildren`. With these tools, you can build complex navigation flows for SPAs.

Exercise 8: Set up routes for an e-commerce app with Home, Products (lazy-loaded), Product Detail (with `:id`), Cart, and Checkout pages. Add a `CanActivate` guard that redirects unauthenticated users from the Checkout page to a Login page. Add a 404 wildcard route. Use `routerLinkActive` to highlight the active nav item. Use query params for product filtering (`?category=electronics&sort=price`).

HTTP & Services

HttpClient Overview

Angular's `HttpClient` service, part of `@angular/common/http`, provides a simplified API for making HTTP requests. It returns typed Observables, supports request/response interception, and includes features like progress events and JSONP.

Setting Up HttpClientModule

Import `HttpClientModule` into your root or feature module:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule],
  // ...
})
export class AppModule { }
```

For standalone apps, use `provideHttpClient()` in the providers array:

```
// main.ts or app.config.ts
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()]
};
```

Making HTTP Requests

GET Request

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

interface Product {
  id: number;
  name: string;
  price: number;
}

@Injectable({ providedIn: 'root' })
export class ProductService {
  private apiUrl = 'https://api.example.com/products';

  constructor(private http: HttpClient) { }

  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>(this.apiUrl);
  }
}
```

POST Request

```
createProduct(product: Product): Observable<Product> {
  return this.http.post<Product>(this.apiUrl, product);
}
```

PUT Request

```
updateProduct(id: number, product: Product): Observable<Product> {
  return this.http.put<Product>(`${this.apiUrl}/${id}`, product);
}
```

DELETE Request

```
deleteProduct(id: number): Observable<void> {  
  return this.http.delete<void>(`${this.apiUrl}/${id}`);  
}
```

HttpParams

Add query parameters to requests:

```
import { HttpParams } from '@angular/common/http';  
  
searchProducts(query: string, page: number): Observable<Product[]> {  
  const params = new HttpParams()  
    .set('q', query)  
    .set('page', page.toString())  
    .set('limit', '20');  
  
  return this.http.get<Product[]>(this.apiUrl, { params });  
}
```

HttpHeaders

Set custom headers for requests:

```
import { HttpHeaders } from '@angular/common/http';  
  
getHeaders(): HttpHeaders {  
  return new HttpHeaders({  
    'Content-Type': 'application/json',  
    'Authorization': `Bearer ${this.auth.getToken()}`,  
    'X-Custom-Header': 'custom-value'  
  });  
}  
  
getSecuredData(): Observable<any> {  
  return this.http.get(this.apiUrl, { headers: this.getHeaders() });  
}
```

Subscribing to Observables

HttpClient methods return cold Observables. The request is made only when you subscribe:

```
this.productService.getProducts().subscribe({
  next: (products) => this.products = products,
  error: (err) => this.errorMessage = 'Failed to load products',
  complete: () => this.loading = false
});
```

Error Handling

Use RxJS operators like `catchError` and `retry` for robust error handling:

```
import { catchError, retry, throwError } from 'rxjs';

getProducts(): Observable<Product[]> {
  return this.http.get<Product[]>(this.apiUrl).pipe(
    retry(2), // Retry failed requests up to 2 times
    catchError(this.handleError)
  );
}

private handleError(error: HttpResponse) {
  let errorMsg = 'An unknown error occurred';
  if (error.error instanceof ErrorEvent) {
    errorMsg = `Client error: ${error.error.message}`;
  } else {
    errorMsg = `Server error: ${error.status} - ${error.message}`;
  }
  console.error(errorMsg);
  return throwError(() => new Error(errorMsg));
}
```

Interceptors

Interceptors examine and transform HTTP requests and responses globally. They are useful for adding auth tokens, logging, caching, or error handling.

Auth Interceptor

```
import { HttpInterceptor, HttpRequest, HttpHandler } from '@angular/common/http';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) { }

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = this.auth.getToken();
    const authReq = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;
    return next.handle(authReq);
  }
}

// Provide the interceptor
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

Logging Interceptor

```
@Injectable()
export class LoggingInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const start = Date.now();
    return next.handle(req).pipe(
      tap(() => console.log(`${req.method} ${req.url} took ${Date.now() - start} ms`))
    );
  }
}
```

File Upload

Upload files using `FormData` with POST:

```
uploadFile(file: File): Observable<any> {
  const formData = new FormData();
  formData.append('file', file, file.name);
  return this.http.post('/api/upload', formData, {
    reportProgress: true,
    observe: 'events'
  });
}
```

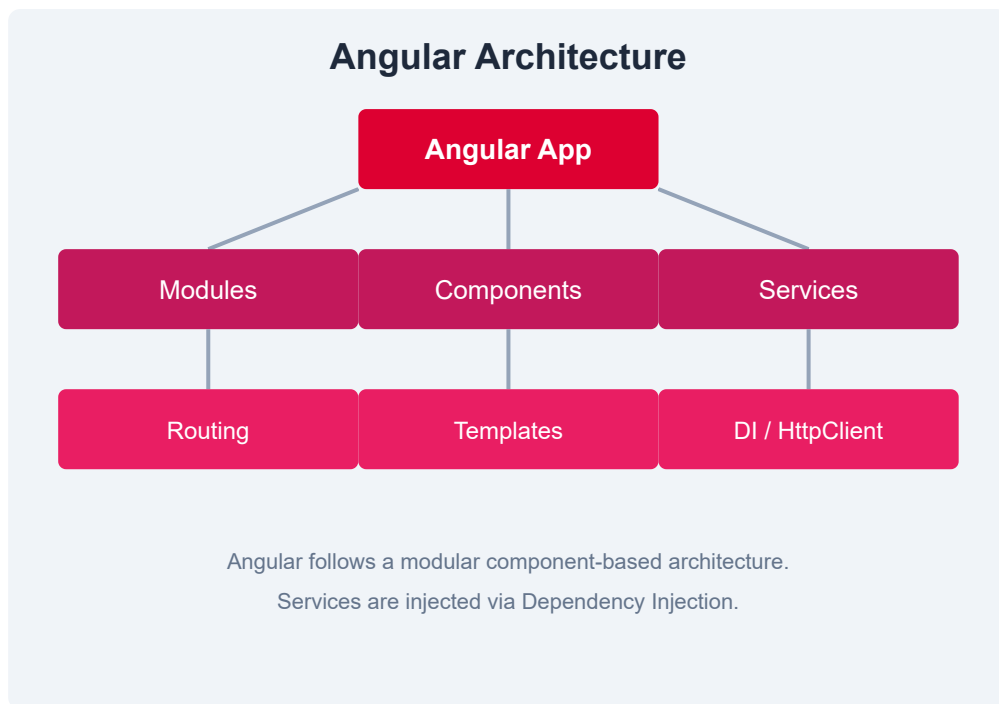
Subscribe with progress tracking:

```
this.uploadService.uploadFile(file).subscribe(event => {
  if (event.type === HttpEventType.UploadProgress) {
    const percent = Math.round(100 * event.loaded / event.total);
    this.progress = percent;
  } else if (event.type === HttpEventType.Response) {
    console.log('Upload complete', event.body);
  }
});
```

JSONP Support

For cross-origin requests that don't support CORS, use `HttpClient.jsonp()`:

```
this.http.jsonp('https://api.example.com/callback?format=jsonp', 'callback')
  .subscribe(data => console.log(data));
```



Summary

You learned how to use Angular's HttpClient for full CRUD operations, configure HttpParams and HttpHeaders, handle errors with catchError and retry, implement request/response interceptors for auth and logging, and handle file uploads with progress tracking. These HTTP patterns are essential for any real-world Angular application.

Exercise 9: Create a `TodoService` with full CRUD operations against JSONPlaceholder (<https://jsonplaceholder.typicode.com/todos>). Implement a logging interceptor that logs all request URLs and durations. Add a retry of 1 for failed requests. Build a `TodoComponent` that displays todos, marks them as complete (PUT), adds new todos (POST), and deletes them (DELETE) with error handling using `catchError`.

Modules

What are NgModules?

An `NgModule` is a container that organizes related components, directives, pipes, and services into cohesive blocks of functionality. Every Angular application has at least one module — the root module. As applications grow, you split them into feature modules, shared modules, and routing modules to maintain clarity and enable lazy loading.

Root Module (AppModule)

The root module is the entry point of the application. It bootstraps the root component and imports essential modules like `BrowserModule` and `HttpClientModule`.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';
import { AppComponent } from './app.component';
import { AppRoutingModule } from './app-routing.module';

@NgModule({
  declarations: [AppComponent],           // Components, directives, pipes belong here
  imports: [BrowserModule, HttpClientModule, AppRoutingModule], // Other modules to import
  providers: [],                          // Services available application-wide
  bootstrap: [AppComponent]              // Root component to bootstrap
})
export class AppModule { }
```

NgModule Metadata Properties

- **declarations** — The components, directives, and pipes that belong to this module. They are private by default and only usable within this module unless exported.
- **imports** — Other modules whose exported declarations are available to this module's templates.
- **exports** — Subset of declarations that can be used in the templates of modules that import this module.
- **providers** — Services registered with the application's root injector (or the module's injector for feature modules).
- **bootstrap** — The root component that Angular creates and inserts into the `index.html` host page (only in root module).

Feature Modules

Feature modules group related functionality for a specific feature area (e.g., Products, Orders, Admin). They help organize code into logical boundaries and enable lazy loading.

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ProductListComponent } from './product-list.component';
import { ProductDetailComponent } from './product-detail.component';
import { ProductsRoutingModule } from './products-routing.module';

@NgModule({
  declarations: [ProductListComponent, ProductDetailComponent],
  imports: [CommonModule, FormsModule, ProductsRoutingModule],
  exports: [ProductListComponent] // Optional: expose if used outside this m
})
export class ProductsModule { }
```

Feature modules should import `CommonModule` instead of `BrowserModule`.
`BrowserModule` should only be imported once — in the root module.
`CommonModule` provides common directives like `ngIf` and `ngFor`.

Shared Modules

Shared modules contain commonly used components, directives, pipes, and pipes that multiple feature modules need. Create a shared module to declare and export reusable pieces.

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { HighlightDirective } from './highlight.directive';
import { TruncatePipe } from './truncate.pipe';
import { LoadingSpinnerComponent } from './loading-spinner.component';

@NgModule({
  declarations: [HighlightDirective, TruncatePipe, LoadingSpinnerComponent],
  imports: [CommonModule, FormsModule],
  exports: [
    HighlightDirective,      // Custom directive
    TruncatePipe,           // Custom pipe
    LoadingSpinnerComponent, // Custom component
    CommonModule,           // Re-export CommonModule so importing modules
    FormsModule             // Re-export FormsModule for template-driven fo
  ]
})
export class SharedModule { }
```

Feature modules then import `SharedModule` to access all the common pieces without duplicating declarations.

Routing Modules

Routing modules separate routing configuration from the feature module itself. This follows Angular's recommended pattern and enables lazy loading easily.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { ProductListComponent } from './product-list.component';
import { ProductDetailComponent } from './product-detail.component';
import { AuthGuard } from '../core/auth.guard';

const routes: Routes = [
  { path: '', component: ProductListComponent },
  { path: ':id', component: ProductDetailComponent, canActivate: [AuthGuard]
  ];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class ProductsRoutingModule { }
```

Module Organization Best Practices

As your application scales, organize modules into layers:

- **CoreModule** — Singleton services, app-wide components (header, footer), guards, and interceptors. Import once in AppModule.
- **SharedModule** — Reusable components, directives, and pipes. Import in every feature module.
- **Feature Modules** — One module per feature area. Lazy-load them via routing.
- **Routing Module** — Separate module for route configuration per feature.

Avoid re-importing services in lazy-loaded feature modules through the module's providers array. Doing so creates a separate instance of the service that is scoped to that module, which can lead to unexpected behavior if a singleton was intended. Use `providedIn: 'root'` on services instead.

 Angular module architecture

Example: AppModule with All Module Types

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { CoreModule } from './core/core.module';
import { SharedModule } from './shared/shared.module';
import { ProductsModule } from './products/products.module';
import { AppRoutingModule } from './app-routing.module';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    CoreModule,           // Singleton services and app shell
    SharedModule,         // Common components, pipes, directives
    ProductsModule,       // Feature module (can be lazy-loaded in production)
    AppRoutingModule      // Root routing – should be last
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Summary

You now understand Angular module architecture: the root module (AppModule), feature modules for organizing feature areas, shared modules for reusable declarations, and routing modules for route configuration. You also learned about NgModule metadata properties (declarations, imports, exports, providers, bootstrap) and best practices for structuring enterprise applications.

Exercise 10: Create a `SharedModule` that exports a `HighlightDirective`, a `SortByPipe`, and a `ModalComponent`. Create a `CoreModule` that provides an `AuthService` (singleton), an auth interceptor, and a `HeaderComponent`. Create a feature `OrdersModule` with its own routing module. Then refactor the `AppModule` to import `CoreModule`, `SharedModule`, and lazy-load `OrdersModule`.

Animations & UI

Angular comes with a powerful animation system and integrates with many UI component libraries. This chapter covers Angular's built-in animation DSL and popular third-party UI frameworks.

Angular Animations

The `@angular/animations` package provides a declarative way to define motion in your application. Import `BrowserAnimationsModule` to enable the animation engine.

```
// app.module.ts
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';

@NgModule({
  imports: [BrowserAnimationsModule]
})
export class AppModule { }
```

trigger & state

Use `trigger` to bind an animation to a template element. Define `state` for each visual phase and `transition` for how to move between them.

```
// fade.animation.ts
import { trigger, state, style, transition, animate } from '@angular/animations';

export const fadeAnimation = trigger('fade', [
  state('void', style({ opacity: 0 })),
  transition(':enter, :leave', [
    animate('300ms ease-in-out')
  ])
]);
```

keyframes

For more complex sequences, use `keyframes` to define intermediate steps within a single transition.

```
trigger('bounce', [  
  transition('* => *', [  
    animate('1s', keyframes([  
      style({ transform: 'translateY(0)', offset: 0 }),  
      style({ transform: 'translateY(-30px)', offset: 0.5 }),  
      style({ transform: 'translateY(0)', offset: 1 })  
    ]))  
  ]) )  
)
```

Angular Material

Angular Material is Google's official component library for Angular, built on top of Material Design. It provides reusable, accessible UI components.

Common Material Components

- **MatButton** — Styled buttons with ripple effects
- **MatCard** — Content containers with elevation
- **MatTable** — Data tables with sorting, filtering, and pagination
- **MatDialog** — Modal dialogs for confirmations or forms
- **MatFormField** — Input fields with labels, hints, and validation

```
import { MatButtonModule } from '@angular/material/button';  
import { MatCardModule } from '@angular/material/card';  
import { MatTableModule } from '@angular/material/table';  
import { MatDialogModule } from '@angular/material/dialog';  
import { MatFormFieldModule } from '@angular/material/form-field';
```

PrimeNG

PrimeNG is a rich set of open-source UI components. It includes advanced widgets like tree tables, charts, editors, and schedules.

```
// app.module.ts
import { TableModule } from 'primeng/table';
import { CalendarModule } from 'primeng/calendar';
import { DropdownModule } from 'primeng/dropdown';

@NgModule({
  imports: [TableModule, CalendarModule, DropdownModule]
})
export class AppModule { }
```

PrimeNG offers both free and premium (Pro) component sets with extensive theme customization.

Bootstrap with Angular

You can use Bootstrap's CSS classes and grid system in Angular projects. For interactive components (modals, tooltips), install `ngx-bootstrap` or `@ng-bootstrap/ng-bootstrap`.

```
npm install bootstrap @ng-bootstrap/ng-bootstrap
```

Icons

Popular icon libraries include Font Awesome and Material Icons. Install the corresponding Angular packages for seamless integration.

```

npm install @fortawesome/angular-fontawesome
npm install @angular/material

// Using Font Awesome
import { FaIconLibrary } from '@fortawesome/angular-fontawesome';
import { faUser } from '@fortawesome/free-solid-svg-icons';

```

Tables with Sorting & Filtering

Combine `MatTable` with `MatSort` and `MatPaginator` for a fully interactive data table.

```

import { MatSort } from '@angular/material/sort';
import { MatPaginator } from '@angular/material/paginator';

@Component({ ... })
export class UsersComponent implements AfterViewInit {
  @ViewChild(MatSort) sort: MatSort;
  @ViewChild(MatPaginator) paginator: MatPaginator;
  dataSource = new MatTableDataSource<User>([]);

  ngAfterViewInit() {
    this.dataSource.sort = this.sort;
    this.dataSource.paginator = this.paginator;
  }
}

```

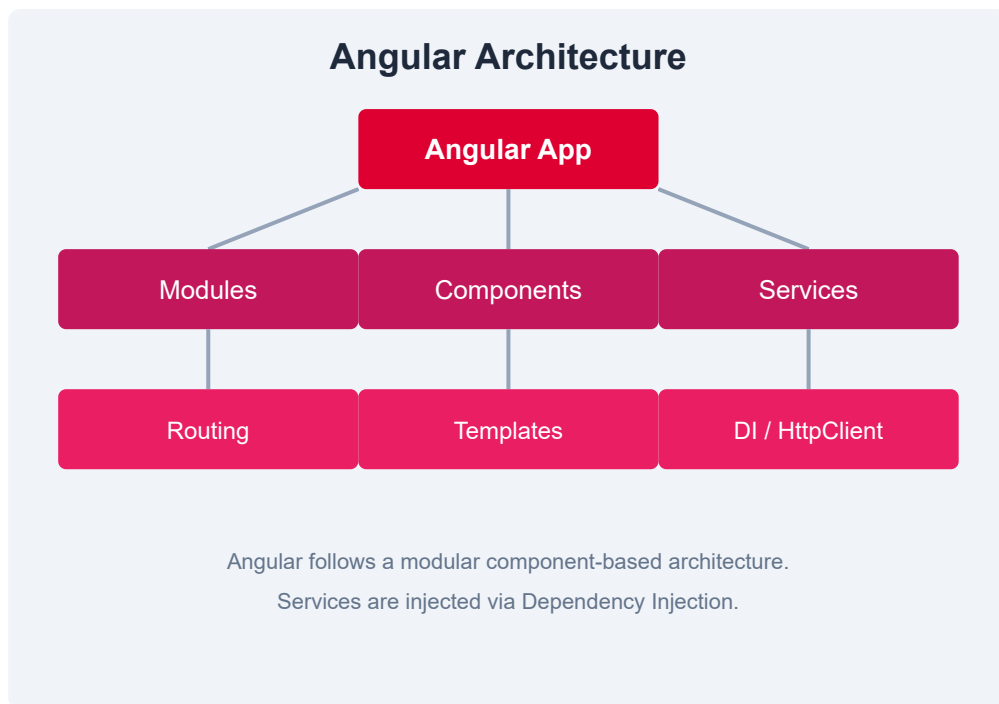
Carousel

Use Angular Material's carousel or PrimeNG's `Carousel` component to display rotating content, banners, or product images.

```

<p-carousel [value]="images" [numVisible]="3" [circular]="true">
  <ng-template let-image pTemplate="item">
    <img [src]="image.src" [alt]="image.alt" />
  </ng-template>
</p-carousel>

```



Dropdowns & Datepickers

Both Material and PrimeNG provide feature-rich dropdown selectors and date pickers with localization support.

Always check the accessibility (a11y) compliance of third-party components. Material and PrimeNG both follow WCAG guidelines.

Practice Task: Create a product card component using MatCard that displays an image, title, description, and price. Add a MatButton that toggles a "favorite" animation using Angular animations (scale and color change).

Progressive Web Apps (PWA)

A Progressive Web App (PWA) delivers an app-like experience through the browser. Angular provides first-class support for building PWAs using service workers and manifest files.

What is a PWA?

PWAs are web applications that use modern browser capabilities to provide features traditionally available only to native apps: offline support, push notifications, install prompts, and fast loading. Key characteristics include:

- **Progressive** — Works for every user, regardless of browser
- **Responsive** — Fits any form factor (desktop, mobile, tablet)
- **Connectivity-independent** — Works offline or on low-quality networks
- **App-like** — Provides a native-app-style interaction
- **Fresh** — Always up-to-date thanks to service workers
- **Safe** — Served via HTTPS to prevent snooping
- **Installable** — Users can add the app to their home screen

Service Workers in Angular

The `@angular/service-worker` package integrates a service worker into your Angular application. Add it using the Angular CLI.

```
ng add @angular/pwa
```

This command sets up the service worker, creates a `manifest.webmanifest` file, and configures `ngsw-config.json`.

ServiceWorkerModule

Register the service worker in your application module:

```
// app.module.ts
import { ServiceWorkerModule } from '@angular/service-worker';
import { environment } from '../environments/environment';

@NgModule({
  imports: [
    ServiceWorkerModule.register('ngsw-worker.js', {
      enabled: environment.production,
      registrationStrategy: 'registerWhenStable:30000'
    })
  ]
})
export class AppModule { }
```

ngsw-config.json

This configuration file defines how the service worker caches resources and handles data requests.

Asset Groups

Asset groups define which static resources (HTML, CSS, JS, images) to cache and how.

```

{
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "updateMode": "lazy",
      "resources": {
        "files": [
          "/favicon.ico",
          "/index.html",
          "/*.css",
          "/*.js"
        ]
      }
    },
    {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "lazy",
      "resources": {
        "files": [
          "/assets/**"
        ]
      }
    }
  ]
}

```

Data Groups

Data groups define caching strategies for API calls and other dynamic data.

```
{
  "dataGroups": [
    {
      "name": "api",
      "urls": ["/api/**"],
      "cacheConfig": {
        "maxSize": 100,
        "maxAge": "1d",
        "timeout": "5s",
        "strategy": "performance"
      }
    }
  ]
}
```

The `strategy` property can be `performance` (cache-first) or `freshness` (network-first).

Caching Strategies

Angular's service worker supports several caching strategies:

- **Prefetch** — Download all resources during installation
- **Lazy** — Download resources on first request
- **Performance** — Serve from cache, fall back to network
- **Freshness** — Try network first, fall back to cache

Install Prompt

Trigger the browser's native install prompt to allow users to add your app to their home screen.

```
// install-prompt.service.ts
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class InstallPromptService {
  private deferredPrompt: any;
  isInstallable = false;

  constructor() {
    window.addEventListener('beforeinstallprompt', (e) => {
      e.preventDefault();
      this.deferredPrompt = e;
      this.isInstallable = true;
    });
  }

  install() {
    this.deferredPrompt?.prompt();
    this.deferredPrompt = null;
    this.isInstallable = false;
  }
}
```

Offline Support

Once the service worker is active and resources are cached, your application can function without a network connection. Monitor online/offline status with Angular's `@HostListener`.

```
@HostListener('window:offline')
onOffline() { this.isOnline = false; }

@HostListener('window:online')
onOnline() { this.isOnline = true; }
```

PWA Manifest

The `manifest.webmanifest` file controls how your app appears when installed. It includes the app name, icons, theme color, and display mode.

```
{
  "name": "Angular PWA",
  "short_name": "AngularPWA",
  "theme_color": "#1976d2",
  "background_color": "#fafafa",
  "display": "standalone",
  "scope": "/",
  "start_url": "/",
  "icons": [
    { "src": "assets/icons/icon-192x192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "assets/icons/icon-512x512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

 Angular PWA service worker flow

Lighthouse Audit

Use Google Lighthouse (built into Chrome DevTools) to audit your PWA. It checks for installability, offline support, performance, and best practices. Aim for a score of 90+ on all categories.

Always test your PWA in an incognito window to ensure the service worker registers fresh. Use Chrome DevTools > Application > Service Workers to debug.

Practice Task: Add the `@angular/pwa` package to an existing Angular project. Configure `ngsw-config.json` to cache all API calls under `/api/**` with a performance strategy. Verify offline functionality using Chrome DevTools.

Testing & Patterns

Testing is a critical part of building reliable Angular applications. This chapter covers unit testing with Jasmine and Karma, along with common design patterns used throughout Angular.

Unit Testing with Jasmine & Karma

Angular CLI scaffolds Jasmine test files alongside your components, services, and other artifacts. Karma is the test runner that executes these tests in a browser environment.

```
ng test
```

This command starts Karma, opens a browser, runs all `*.spec.ts` files, and reports results.

TestBed

`TestBed` is the primary API for configuring Angular testing modules. Use it to create a testing environment that mimics your application module.

```
beforeEach(async () => {  
  await TestBed.configureTestingModule({  
    declarations: [MyComponent],  
    imports: [HttpClientTestingModule],  
    providers: [MyService]  
  }).compileComponents();  
});
```

ComponentFixture & detectChanges

A `ComponentFixture` wraps the component for testing. Call `detectChanges()` to trigger change detection.

```
it('should display the title', () => {
  const fixture = TestBed.createComponent(MyComponent);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;
  expect(compiled.querySelector('h1')?.textContent).toContain('Welcome');
});
```

debugElement

`debugElement` provides a type-safe way to query the component template and inspect its properties.

```
it('should have a submit button', () => {
  const fixture = TestBed.createComponent(MyComponent);
  const debugEl = fixture.debugElement;
  const btn = debugEl.query(By.css('button[type="submit"]'));
  expect(btn).toBeTruthy();
});
```

Testing Services

Use `HttpClientTestingModule` and `HttpTestingController` to test HTTP-based services without making real network requests.

```

let httpTestingController: HttpTestingController;
let service: DataService;

beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [HttpClientTestingModule],
    providers: [DataService]
  });
  httpTestingController = TestBed.inject(HttpTestingController);
  service = TestBed.inject(DataService);
});

it('should return expected data', () => {
  const testData = { id: 1, name: 'Test' };
  service.getData().subscribe(data => {
    expect(data).toEqual(testData);
  });
  const req = httpTestingController.expectOne('/api/data');
  req.flush(testData);
});

```

Lazy Loading with loadChildren

Lazy loading delays module loading until the user navigates to that route, improving initial bundle size.

```

const routes: Routes = [
  { path: 'dashboard', loadChildren: () => import('./dashboard/dashboard.module')
  };

```

Singleton Pattern (providedIn: 'root')

Marking a service with `providedIn: 'root'` ensures only one instance exists across the entire application — the singleton pattern.

```

@Injectable({ providedIn: 'root' })
export class SharedService {
  private data = new BehaviorSubject<string>('initial');
  data$ = this.data.asObservable();

  updateData(value: string) {
    this.data.next(value);
  }
}

```

Observer Pattern (Subject, BehaviorSubject)

RxJS Subjects implement the observer pattern. `Subject` multicasts values to multiple subscribers. `BehaviorSubject` also emits the current value to new subscribers.

```

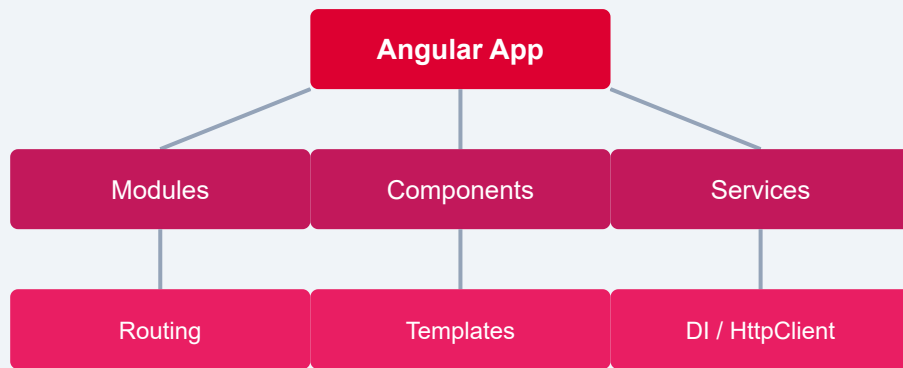
const subject = new Subject<number>();
subject.subscribe(v => console.log('A:', v));
subject.next(1);
subject.next(2);

const behaviorSubject = new BehaviorSubject<number>(0);
behaviorSubject.subscribe(v => console.log('B:', v));
behaviorSubject.next(1);

```

Use `BehaviorSubject` over `Subject` when new subscribers need to know the latest value immediately.

Angular Architecture



Angular follows a modular component-based architecture.
Services are injected via Dependency Injection.

Practice Task: Write a unit test for a service that fetches user data from `/api/users`. Mock the HTTP call using `HttpTestingController`. Then write a component test that verifies the user list renders correctly in the template.

Libraries & Advanced

Angular integrates deeply with RxJS for reactive programming and introduces Signals for fine-grained reactivity. This chapter explores advanced libraries, authentication, internationalization, and standalone components.

RxJS

RxJS (Reactive Extensions for JavaScript) is the foundation of Angular's asynchronous and event-based programming model.

Core Types

- **Observable** — Represents a stream of values over time
- **Subject** — An Observable that can emit new values via `.next()`
- **BehaviorSubject** — A Subject that holds the current value and emits it to new subscribers

Operators

Operators transform, filter, and combine Observable streams.

```
import { of, fromEvent } from 'rxjs';
import { map, filter, debounceTime, switchMap } from 'rxjs/operators';

of(1, 2, 3, 4, 5).pipe(
  filter(x => x % 2 === 0),
  map(x => x * 10)
).subscribe(console.log); // 20, 40

// Search input with debounce
fromEvent(input, 'input').pipe(
  debounceTime(300),
  switchMap(() => this.searchService.search(input.value))
).subscribe(results => this.results = results);
```

switchMap cancels the previous inner observable when a new value arrives — ideal for typeahead searches.

Angular Signals

Signals provide a new reactive primitive for Angular. They are simpler than RxJS for state management and integrate deeply with change detection.

```
import { signal, computed, effect, input, output } from '@angular/core';

const count = signal(0);
const doubled = computed(() => count() * 2);

effect(() => console.log('Count:', count()));

// In a component
export class CounterComponent {
  value = input(0);
  changed = output<number>();

  increment() {
    this.value.update(v => v + 1);
    this.changed.emit(this.value());
  }
}
```

Authentication (JWT)

JSON Web Tokens (JWT) are commonly used for stateless authentication. Store tokens in `localStorage` and attach them via HTTP interceptors.

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    if (token) {
      req = req.clone({
        headers: req.headers.set('Authorization', `Bearer ${token}`)
      });
    }
    return next.handle(req);
  }
}

@Injectable({ providedIn: 'root' })
export class AuthService {
  private user = signal<User | null>(null);
  isLoggedIn = computed(() => this.user() !== null);

  login(credentials: Credentials) {
    return this.http.post<{ token: string }>('/api/login', credentials).pipe(
      tap(res => {
        localStorage.setItem('token', res.token);
        this.user.set(this.decodeToken(res.token));
      })
    );
  }
}
```

Route Guards

Protect routes using `CanActivate` guard.

```

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (!this.auth.isLoggedIn()) {
      this.router.navigate(['/login']);
      return false;
    }
    return true;
  }
}

```

Internationalization (i18n)

Use `@angular/localize` to translate your application into multiple languages.

```

ng add @angular/localize

// In template
<h1 i18n="Greeting|An introduction greeting">Hello!</h1>

// Extract messages
ng extract-i18n --output-path src/locale

```

Standalone Components

Standalone components eliminate the need for `NgModule`. Use `standalone: true` and `bootstrapApplication`.

```
@Component({
  standalone: true,
  imports: [CommonModule, RouterModule],
  template: `<h1>Standalone!</h1>`
})
export class AppComponent {}

// main.ts
bootstrapApplication(AppComponent, {
  providers: [provideHttpClient(), provideRouter(routes)]
});
```

Web Workers

Run CPU-intensive tasks off the main thread with web workers.

```
const worker = new Worker(new URL('./app.worker', import.meta.url));
worker.postMessage({ data: heavyArray });
worker.onmessage = ({ data }) => console.log(data);
```

SSR with Angular Universal

Angular Universal enables server-side rendering for better SEO and initial load performance.

```
ng add @nguniversal/express-engine
```

 Angular advanced libraries

Practice Task: Refactor a routed Angular module into standalone components. Replace the main module bootstrap with `bootstrapApplication` and configure the router provider. Add an authentication guard to protect a dashboard route.

Advanced Concepts

This chapter dives into Angular's advanced architecture: server-side rendering, the Ivy compiler, build systems, and strategies for upgrading and optimizing production applications.

Server-Side Rendering with Angular Universal

Angular Universal renders your application on the server, sending fully rendered HTML to the client. This improves SEO, social media previews, and perceived performance.

```
ng add @nguniversal/express-engine
```

This generates a server-side bundle and an Express server that handles both rendering and API requests.

platform-server

The `platform-server` package renders Angular applications on the server. It replaces `platform-browser-dynamic` in the server bundle.

```
// main.server.ts
import { enableProdMode } from '@angular/core';
import { environment } from '../environments/environment';

if (environment.production) enableProdMode();

export { AppServerModule } from '../app/app.server.module';
```

TransferState

`TransferState` passes data from the server to the client to avoid re-fetching on initial load.

```
import { TransferState, makeStateKey } from '@angular/platform-browser';

const USERS_KEY = makeStateKey<User[]>('users');

// Server side
constructor(private state: TransferState) {}
this.http.get<User[]>('/api/users').subscribe(users => {
  this.state.set(USERS_KEY, users);
});

// Client side
const cached = this.state.get(USERS_KEY, null as any);
if (cached) {
  this.users = cached;
} else {
  // fetch from API
}
```

Use `TransferState` to eliminate the flash of loading spinners when the client re-hydrates.

Ivy Compiler

Ivy is Angular's next-generation compilation and rendering pipeline, introduced as the default in Angular 9. It produces smaller bundles, faster compilation, and improved debugging.

- **Locality** — Each component is compiled independently
- **Tree-shaking** — Unused code is eliminated at the component level
- **Improved type-checking** — Extended template type checking
- **Faster re-builds** — Incremental compilation for development

Compilation Modes

Angular supports two compilation modes via `angularCompilerOptions`:

```
// tsconfig.json
{
  "angularCompilerOptions": {
    "compilationMode": "full" // or "partial" for library authors
  }
}
```

full — Used for applications, produces fully compiled code. **partial** — Used for libraries, generates Angular-compatible output for consumers.

Bazel Build System

Bazel is Google's build system that handles large-scale monorepos with incremental and parallel builds. Angular libraries can be built with Bazel for faster CI pipelines.

```
ng new my-app --collection=@angular/bazel
```

Reactive Programming Patterns

Combine RxJS with Angular Signals for a powerful reactive architecture:

```
// Convert Observable to Signal
toSignal(this.http.get<User[]>('/api/users'));

// Combine multiple streams
const vm = computed(() => ({
  users: this.users(),
  loading: this.loading(),
  error: this.error()
}));
```

Backward Compatibility (ngUpgrade)

Use the `@angular/upgrade` package to run AngularJS (1.x) and Angular side-by-side in a hybrid application, enabling a gradual migration.

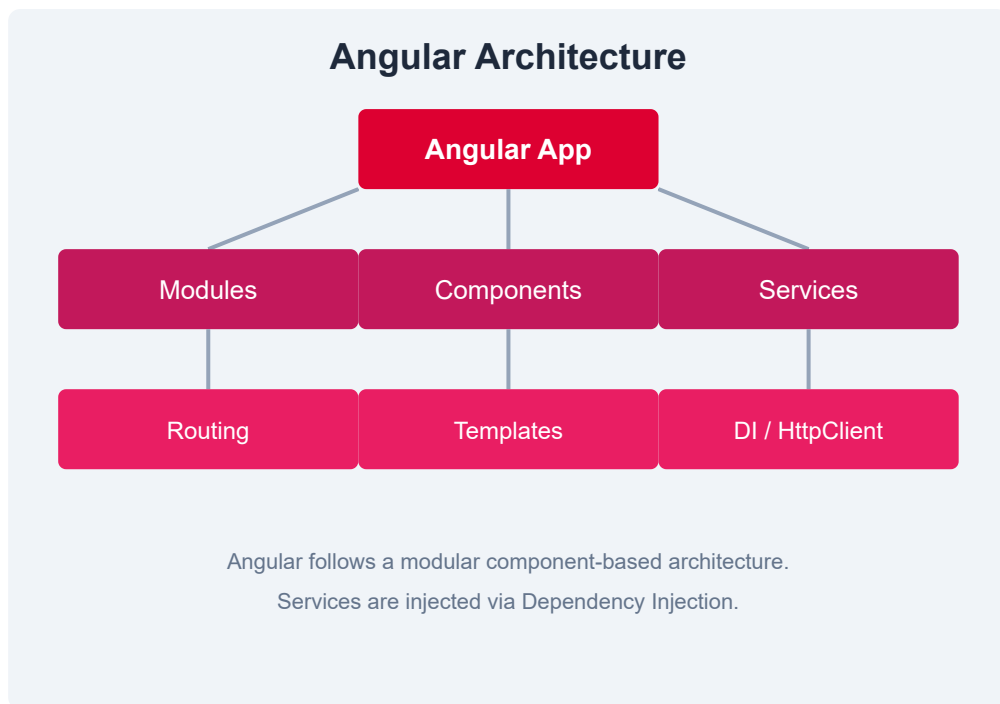
```
import { UpgradeModule } from '@angular/upgrade/static';

// Bootstrap AngularJS inside Angular
@NgModule({
  imports: [UpgradeModule]
})
export class AppModule {
  ngDoBootstrap() { /* handled by UpgradeModule */ }
}
```

Differential Loading

Angular CLI automatically generates two bundles: one for modern browsers (ES2017+) and one for legacy browsers (ES5). The browser loads the appropriate version.

```
<script src="runtime-es2015.js" type="module"></script>
<script src="runtime-es5.js" nomodule></script>
```



Practice Task: Add Angular Universal to an existing Angular project. Configure `TransferState` to pass user data from server to client. Verify that the initial page load shows content immediately without a loading spinner.

Tools

The Angular CLI (Command Line Interface) is the primary tool for scaffolding, developing, and building Angular applications. This chapter covers CLI commands, configuration, decorators, lifecycle hooks, and production deployment.

Angular CLI Commands

The Angular CLI simplifies every phase of development. Below are the most important commands:

ng generate

Scaffold components, directives, pipes, services, modules, and more.

```
ng generate component dashboard
ng generate directive highlight
ng generate pipe filter
ng generate service data
ng generate module admin --route admin --module app.module
ng generate class models/user
ng generate interface models/user
ng generate guard auth
```

ng serve

Start a development server with live reload and source maps.

```
ng serve --open --port 4200 --hmr
```

ng build

Compile the application into output files in the `dist/` folder.

```
ng build --prod --output-path docs --base-href /my-app/
```

ng test & ng e2e

Run unit tests with Karma and end-to-end tests with Protractor or Cypress.

```
ng test --watch=false --code-coverage  
ng e2e
```

ng deploy

Deploy to various cloud providers with a single command.

```
ng deploy --base-href=/my-app/
```

angular.json Configuration

The `angular.json` file defines the workspace structure, project configurations, build options, and environments.

```

{
  "projects": {
    "my-app": {
      "architect": {
        "build": {
          "configurations": {
            "production": {
              "optimization": true,
              "outputHashing": "all",
              "sourceMap": false,
              "extractCss": true,
              "namedChunks": false,
              "aot": true,
              "extractLicenses": true,
              "vendorChunk": false,
              "buildOptimizer": true,
              "budgets": [
                { "type": "initial", "maximumWarning": "2mb" },
                { "type": "anyComponentStyle", "maximumWarning": "6kb" }
              ]
            }
          }
        }
      }
    }
  }
}

```

Decorators & Metadata

Decorators add metadata to classes, methods, and properties. They are the building blocks of Angular's declarative syntax.

```

@Component({ selector: 'app-root', templateUrl: './app.component.html' })
@Directive({ selector: '[appHighlight]' })
@Pipe({ name: 'filter' })
@Injectable({ providedIn: 'root' })
@Input() user: User;
@Output() clicked = new EventEmitter<void>();

```

Lifecycle Hooks Reference

Angular components and directives have lifecycle hooks that let you tap into key moments:

- `ngOnChanges` — Called when input properties change
- `ngOnInit` — Called once after the first `ngOnChanges`
- `ngDoCheck` — Called during every change detection run
- `ngAfterContentInit` — After content projection
- `ngAfterContentChecked` — After every content check
- `ngAfterViewInit` — After the view and child views are initialized
- `ngAfterViewChecked` — After every view check
- `ngOnDestroy` — Cleanup before Angular destroys the component

Error Handling

Override Angular's `ErrorHandler` to provide global error handling.

```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any) {
    console.error('Global error:', error);
    // Send to logging service
  }
}

// app.module.ts
providers: [{ provide: ErrorHandler, useClass: GlobalErrorHandler }]
```

Building for Production

Use `--prod` to enable AOT compilation, tree-shaking, minification, and dead code elimination.

```
ng build --prod
```

This produces optimized bundles in `dist/`. Deploy these files to any static hosting or cloud platform.

 Angular CLI tools diagram

Practice Task: Use the Angular CLI to generate a new Angular application. Configure `angular.json` to set a budget warning of 1 MB for the initial bundle. Build the application in production mode and inspect the output in the `dist/` folder.

Additional Concepts

This chapter provides a quick start guide, curated learning resources, study plans, interview preparation tips, and a glossary to help you continue your Angular journey.

Quick Start Guide

Get started with Angular in five steps:

```
# 1. Install Angular CLI
npm install -g @angular/cli

# 2. Create a new project
ng new my-first-app --routing --style=scss

# 3. Navigate to the project
cd my-first-app

# 4. Start the development server
ng serve --open

# 5. Generate your first component
ng generate component hello-world
```

Your application is now running at `http://localhost:4200`. Edit `src/app/hello-world/hello-world.component.ts` to customize it.

Useful Resources

- **angular.io** — Official Angular documentation with tutorials, guides, and API references
- **angular-university.io** — In-depth video courses on Angular, RxJS, and NgRx

- **ng-conf** — Annual Angular conference with recorded talks and workshops
- **Angular Blog** — Official blog with release notes, case studies, and team updates
- **Stack Overflow** — Active community for troubleshooting and best practices

Discussion Forums

Join the Angular community on these platforms:

- **Angular Discord** — Real-time chat with Angular developers worldwide
- **r/Angular2** — Reddit community for news, questions, and showcases
- **GitHub Discussions** — Official Angular repository for feature requests and feedback

8-Week Study Plan

Follow this plan to go from beginner to productive Angular developer:

- **Week 1-2:** TypeScript, components, templates, data binding
- Directives, pipes, services, dependency injection
- Routing, forms (template-driven & reactive)
- RxJS, HTTP client, error handling
- State management (Signals, NgRx), guards
- Testing (Jasmine/Karma), Angular Material
- PWAs, SSR, production build & deployment

Interview Prep Questions

Common Angular interview questions:

1. What is the difference between `*ngIf` and `[hidden]` ?
2. How does Angular's change detection work?
3. Explain the difference between template-driven and reactive forms.
4. What is a `Subject` and how does it differ from a `BehaviorSubject` ?
5. What are Angular lifecycle hooks? List them in order.

6. How do you implement lazy loading?
7. What is the Ivy compiler and what benefits does it bring?
8. How does dependency injection work in Angular?
9. What are standalone components and when should you use them?
10. How do you optimize an Angular application for production?

Resume Builder Tips

When listing Angular on your resume, highlight specific achievements:

- Mention the version(s) of Angular you have used
- Describe the architecture (standalone vs module-based)
- Include state management (Signals, NgRx, Akita)
- Mention testing coverage and tools used
- Showcase performance optimizations (lazy loading, SSR)

Glossary of Terms

- **AOT** — Ahead-of-Time compilation, compiles templates at build time
- **DI** — Dependency Injection, Angular's way of providing dependencies
- **NgModule** — A class that groups components, directives, pipes, and services
- **Standalone** — A component/directive/pipe that does not belong to an NgModule
- **Signal** — A reactive primitive that tracks state and notifies consumers
- **RxJS** — Reactive Extensions for JavaScript, used for asynchronous streams

Q&A

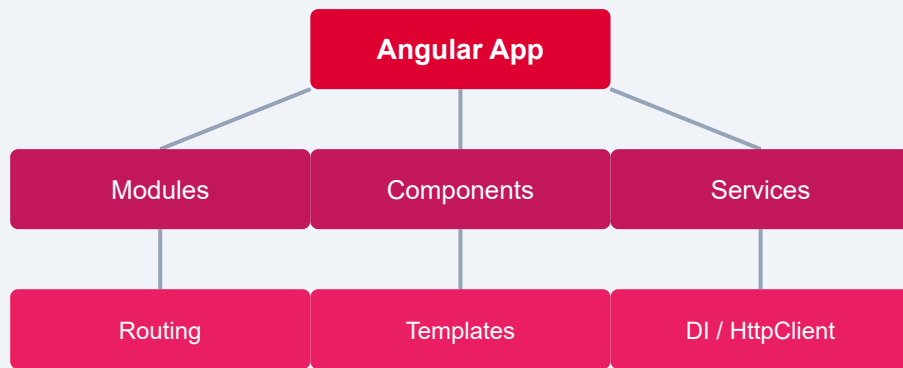
Q: Should I use NgModules or standalone components for new projects?

A: Angular team recommends standalone components for new projects. NgModules are still supported but considered legacy.

Q: What is the best way to manage state in Angular?

A: For small apps, Signals or a simple service with BehaviorSubject. For larger apps, consider NgRx or Akita.

Angular Architecture



Angular follows a modular component-based architecture.

Services are injected via Dependency Injection.

Practice Task: Create a personal study schedule based on the 8-week plan. For each week, create a mini Angular project (e.g., a component library, a form-based app, a dashboard) that reinforces the concepts covered.

Design Patterns in Angular

Design patterns provide reusable solutions to common programming challenges. Angular's architecture naturally implements several classic patterns, and understanding them will help you write cleaner, more maintainable code.

Singleton Pattern

The singleton pattern ensures a class has only one instance. Angular's dependency injection system makes this the default behavior for services.

```
// A singleton service shared across the entire app
@Injectable({ providedIn: 'root' })
export class AppStateService {
  private state = signal<AppState>({ theme: 'light', sidebar: true });

  getState() { return this.state.asReadonly(); }
  updateState(partial: Partial<AppState>) {
    this.state.update(s => ({ ...s, ...partial }));
  }
}

// Both components share the same instance
@Component({ ... })
export class SidebarComponent {
  constructor(private state: AppStateService) {}
}

@Component({ ... })
export class ThemeToggleComponent {
  constructor(private state: AppStateService) {}
}
```

When a service is provided at the root level, Angular creates exactly one instance and injects it into every consumer.

Observer Pattern

RxJS is built on the observer pattern. A **Subject** (observable) maintains a list of subscribers (observers) and notifies them of state changes.

```
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private notifications = new Subject<Notification>();
  notifications$ = this.notifications.asObservable();

  notify(message: string, type: 'success' | 'error' | 'info') {
    this.notifications.next({ message, type, timestamp: Date.now() });
  }
}

// Component A emits
this.notificationService.notify('Saved!', 'success');

// Component B listens
this.notificationService.notifications$.subscribe(n => {
  this.showToast(n.message, n.type);
});
```

Factory Pattern

The factory pattern creates objects without specifying the exact class. In Angular, use **useFactory** in providers or dynamic component creation for this purpose.

```

// Dynamic component factory
@Directive({ selector: '[adHost]' })
export class AdDirective {
  constructor(public viewContainerRef: ViewContainerRef) {}
}

export class AdService {
  getAds() {
    return [
      { component: HeroProfileComponent, data: { name: 'Angular' } },
      { component: HeroJobComponent, data: { title: 'Framework' } }
    ];
  }
}

// Load components dynamically
loadComponent(ad: AdItem) {
  const componentRef = this.adHost.viewContainerRef
    .createComponent(ad.component);
  componentRef.instance.data = ad.data;
}

```

Strategy Pattern

The strategy pattern defines a family of algorithms and makes them interchangeable. Use dependency injection to select strategies at runtime.

```
// Strategy interface
export interface ValidationStrategy {
  validate(value: any): { valid: boolean; error?: string };
}

// Concrete strategies
export class EmailStrategy implements ValidationStrategy {
  validate(value: string) {
    const valid = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/\\.test(value);
    return { valid, error: valid ? undefined : 'Invalid email' };
  }
}

export class RequiredStrategy implements ValidationStrategy {
  validate(value: any) {
    return { valid: !!value, error: value ? undefined : 'Required' };
  }
}

// Use via DI
@Injectable()
export class ValidatorService {
  constructor(private strategies: ValidationStrategy[]) {}

  validate(value: any, type: string) {
    const strategy = this.strategies.find(s => s.constructor.name === type);
    return strategy?.validate(value);
  }
}
```

Dependency Injection Patterns

Angular's DI system supports a hierarchy of injectors. You can control service visibility at different levels:

- **@Injectable({ providedIn: 'root' })** — Application-wide singleton
- **providers: [] on @Component** — New instance per component and its children
- **providers: [] on @NgModule** — Shared across the module (lazy modules get their own injector)

```
@Component({
  selector: 'app-user-list',
  providers: [UserService] // New instance for this component tree
})
export class UserListComponent { }
```

Facade Pattern

A facade provides a simplified interface to a complex subsystem. In Angular, service facades wrap multiple services to expose a clean API.

```
@Injectable({ providedIn: 'root' })
export class DashboardFacade {
  constructor(
    private userService: UserService,
    private analyticsService: AnalyticsService,
    private notificationService: NotificationService
  ) {}

  // Simplified API
  loadDashboardData(): Observable<DashboardData> {
    return forkJoin({
      users: this.userService.getUsers(),
      stats: this.analyticsService.getStats()
    }).pipe(
      tap(() => this.notificationService.notify('Dashboard loaded', 'info'))
    );
  }
}
```

Use facades to reduce coupling between your components and the underlying service layer.

 Angular design patterns

Practice Task: Implement a logger service using the strategy pattern. Create two strategies: `ConsoleLogger` and `FileLogger`. Use Angular's DI to inject the correct strategy based on the environment (development vs production).

Practice

Put your Angular skills to the test with hands-on exercises, coding challenges, and full project ideas. Build real applications and solidify your understanding of Angular concepts.

Exercises

Exercise 1: Todo App

Build a complete todo application with the following features:

- Add, edit, and delete tasks
- Mark tasks as complete/incomplete
- Filter tasks (all, active, completed)
- Persist tasks in local storage
- Use reactive forms for task creation

```
// Todo model
export interface Todo {
  id: number;
  title: string;
  completed: boolean;
  createdAt: Date;
}

// Core requirements
@Component({ ... })
export class TodoComponent {
  // Use signal for state
  todos = signal<Todo[]>([]);
  filter = signal<'all' | 'active' | 'completed'>('all');

  // Computed derived state
  filteredTodos = computed(() => {
    const currentFilter = this.filter();
    return this.todos().filter(todo => {
      if (currentFilter === 'active') return !todo.completed;
      if (currentFilter === 'completed') return todo.completed;
      return true;
    });
  });
}
```

Exercise 2: Dashboard

Create a data dashboard that displays statistics and charts:

- Fetch data from a mock API using `HttpClient`
- Display data in tables with sorting and filtering
- Add a chart (use Chart.js or ngx-charts)
- Implement a date range filter
- Use Angular Material for the UI

Exercise 3: Authentication System

Implement a full authentication flow:

- Login and registration forms with validation

- JWT token management (store, refresh, intercept)
- Route guards to protect private pages
- User profile page (view and edit)
- Logout with token cleanup

Coding Challenges

- **Debounced Search:** Build a search input that uses `debounceTime` and `switchMap` to query an API
- **Infinite Scroll:** Implement lazy loading of items when the user scrolls to the bottom
- **Drag & Drop:** Use Angular CDK to create a draggable list with reorder support
- **Multi-Step Form:** Create a wizard with reactive form validation across steps
- **Real-Time Clock:** Use RxJS `interval` and `map` to display a live clock

Full Bootcamp-Style Curriculum

This curriculum mirrors what you would find in a professional Angular bootcamp. Each module builds on the previous one.

1. TypeScript fundamentals, Angular CLI, components, templates
2. Directives, pipes, data binding, event handling
3. Services, dependency injection, HTTP client
4. Routing, lazy loading, route guards
5. Reactive forms, template-driven forms, validation
6. RxJS deep dive, state management with Signals
7. Angular Material, responsive design, accessibility
8. Testing, CI/CD, production deployment

Mini Project Ideas

Weather App

Build a weather application that fetches data from the OpenWeatherMap API. Show current conditions, forecasts, and allow users to search by city. Use signals for state management and Angular Material for the UI.

```
@Injectable({ providedIn: 'root' })
export class WeatherService {
  private apiKey = environment.weatherApiKey;
  private baseUrl = 'https://api.openweathermap.org/data/2.5';

  getCurrentWeather(city: string): Observable<WeatherData> {
    return this.http.get<WeatherData>(`${this.baseUrl}/weather`, {
      params: { q: city,appid: this.apiKey, units: 'metric' }
    });
  }
}
```

Contact Manager

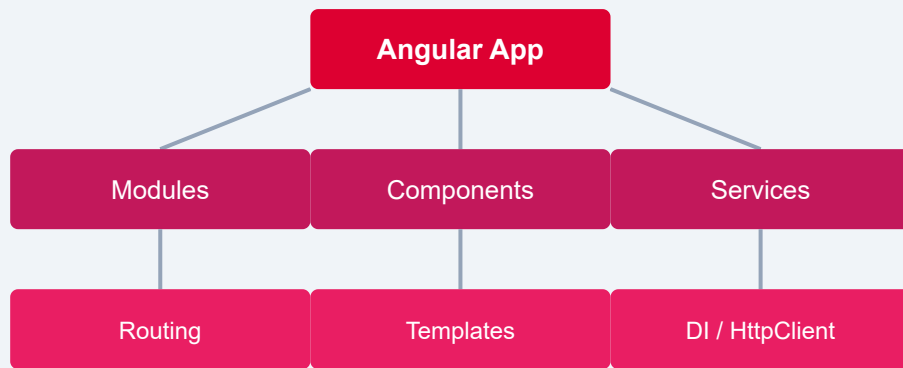
Create a contact management app with CRUD operations, search, filtering, and grouping. Store data using a service with BehaviorSubject. Implement a master-detail layout.

Blog Platform

Build a simple blog with post creation, editing, categories, and comments. Use Angular Universal for SSR to improve SEO. Implement a rich text editor (Quill or TinyMCE) for writing posts.

For all mini projects, start with a minimal viable product and iteratively add features. This mirrors real-world development practices.

Angular Architecture



Angular follows a modular component-based architecture.

Services are injected via Dependency Injection.

Practice Task: Choose one of the three mini projects above (Weather App, Contact Manager, or Blog Platform). Build it from scratch using the Angular CLI. Deploy it to a free hosting service (GitHub Pages, Vercel, or Firebase) and share the URL.

References

This final chapter serves as a comprehensive reference for Angular's core API, DOM abstractions, and key modules. Use it as a quick lookup guide when building your applications.

Angular Core API

The `@angular/core` package provides the foundational building blocks of any Angular application.

Decorators

- **@Component** — Declares a component with selector, template, styles, and change detection
- **@Directive** — Creates a structural or attribute directive
- **@Pipe** — Transforms template values
- **@Injectable** — Marks a class as available for dependency injection
- **@NgModule** — Groups related components, directives, pipes, and services
- **@Input** — Binds a property to a parent template expression
- **@Output** — Emits custom events to the parent
- **@HostBinding** — Binds a property to the host element
- **@HostListener** — Listens to events on the host element
- **@ViewChild** / **@ViewChildren** — Queries child elements or components
- **@ContentChild** / **@ContentChildren** — Queries projected content

Core Classes

- **Renderer2** — Safe DOM manipulation abstraction
- **ElementRef** — Reference to the host element
- **ViewContainerRef** — Container for dynamically created views

- **ComponentFactoryResolver** — Resolves component factories for dynamic creation
- **ChangeDetectorRef** — Controls change detection for a view
- **NgZone** — Manages execution outside Angular's zone

DOM Reference (Renderer2)

Use `Renderer2` instead of directly manipulating the DOM. It ensures compatibility across different rendering environments (browser, server, web worker).

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  constructor(private renderer: Renderer2, private el: ElementRef) {}

  @HostListener('mouseenter')
  onMouseEnter() {
    this.renderer.setStyle(this.el.nativeElement, 'background', 'yellow');
  }

  @HostListener('mouseleave')
  onMouseLeave() {
    this.renderer.removeStyle(this.el.nativeElement, 'background');
  }

  setColor(color: string) {
    this.renderer.setStyle(this.el.nativeElement, 'color', color);
  }
}
```

API Reference by Module

Router (@angular/router)

- `RouterModule.forRoot(routes)` — Configures root routes
- `RouterLink` / `RouterLinkActive` — Template navigation directives
- `RouterOutlet` — Renders matched route component
- `ActivatedRoute` — Access route parameters, query params, data

- `CanActivate` , `CanDeactivate` , `Resolve` — Route guards

Forms (@angular/forms)

- `FormGroup` , `FormControl` , `FormArray` — Reactive form classes
- `Validators` — Built-in validation (required, email, minLength, etc.)
- `FormsModule` — Template-driven forms
- `ReactiveFormsModule` — Reactive forms

HTTP (@angular/common/http)

- `HttpClient` — HTTP request methods (get, post, put, delete)
- `HttpInterceptor` — Intercept and modify requests/responses
- `HttpParams` — Build URL query parameters
- `HttpHeaders` — Build HTTP headers

Compiler (@angular/compiler)

- `compileComponent` , `compileNgModule` — JIT compilation API
- `Compiler` — Programmatic access to the Angular compiler

Animations (@angular/animations)

- `trigger` , `state` , `style` , `transition` , `animate` , `keyframes`
- `group` , `sequence` , `query` , `stagger` — Complex animation helpers

Common (@angular/common)

- `NgIf` , `NgFor` , `NgSwitch` , `NgClass` , `NgStyle` — Built-in directives
- `AsyncPipe` , `DatePipe` , `UpperCasePipe` , `JsonPipe` — Built-in pipes
- `Location` , `PlatformLocation` — URL and platform utilities

Platform Browser (@angular/platform-browser)

- `BrowserModule` — Core module for browser applications
- `DomSanitizer` — Sanitizes HTML, URLs, and styles for safe binding
- `TransferState` — Passes data from server to client

Platform Browser Dynamic (@angular/platform-browser-dynamic)

- `platformBrowserDynamic` — JIT bootstrapping for development
- `JitCompilerFactory` — JIT compiler configuration



Angular API reference

Bookmark the official Angular API documentation at angular.io/api for the most up-to-date and comprehensive reference.

Practice Task: Create a reference card (cheat sheet) for yourself that lists the top 20 Angular APIs you use most frequently. Include the import path, a brief description, and a one-line code example for each.